

一、 产品设计

1.1 用户角色与核心场景

用户角色与核心场景	
角色	核心场景
客户/买家 (Customer/Buyer)	浏览商品列表、查看商品详情、将商品加入购物车、修改购物车商品数量、提交订单、选择支付方式并完成支付、查看个人订单列表和订单详情、追踪订单发货状态、确认收货。
运营人员 (Operations Staff)	管理商品（创建、上架/下架、编辑信息）。管理 SKU 和库存（调整库存数量）。处理订单（查看订单列表、标记订单为已发货、更新物流单号）。处理简单的售后（同意/拒绝退款申请）。
平台管理员 (Platform Admin)	管理运营人员账号及权限。查看核心业务数据（如 GMV、订单量、转化率）。处理运营人员无法解决的异常订单（如强制取消、退款）。

1.2 MVP 功能清单与优先级

MVP 功能清单与优先级			
模块	功能	优先级	原因
客户端	用户注册/登录	P0	识别用户身份，关联订单和购物车数据。
客户端	商品列表/详情页	P0	用户浏览和获取商品信息的核心入口。
客户端	购物车	P1	提升批量购买体验，虽是 P1 但对转化率很重要。P0 可用“立即购买”替代。
客户端	提交订单	P0	核心交易流程的起点。
客户端	集成支付（模拟/真实）	P0	完成交易闭环。初期可集成沙箱支付或模拟支付成功/失败。
客户端	订单列表/详情	P0	用户查看订单状态，是售后服务的基础。
客户端	取消订单	P1	提升用户体验，减少无效订单。
客户端	商品搜索/分类筛选	P2	商品少时非必需，后期可迭代。
管理端	商品管理（增删改查）	P0	必须要有商品才能卖。核心是创建、编辑、上/下架。
管理端	SKU/库存管理	P0	管理不同规格商品的数量，防止超卖。
管理端	订单管理（列表/详情）	P0	查看订单，是处理发货和售后的基础。

管理端	发货管理	P0	更新订单状态为“已发货”，完成交易履约。
管理端	用户管理	P2	MVP 阶段用户量小，可由运营直接在数据库操作或简易页面管理。
管理端	数据看板	P2	初期依赖数据库查询，独立看板非核心路径。

1.3 核心流程设计

流程一：客户下单与支付流程

A. 正常流程 (Normal Flow):

- 1) 用户在购物车或商品详情页点击“立即购买”。
- 2) 系统跳转至“确认订单”页，展示商品信息、数量、金额、收货地址。
- 3) 用户选择/填写收货地址，选择支付方式（如模拟支付），点击“提交订单”。
- 4) 后端创建订单，状态为 `PENDING_PAYMENT`，并锁定库存（预扣）。
- 5) 订单创建成功，跳转至“支付页面”。
- 6) 用户确认金额，点击“立即支付”，调用支付接口。
- 7) 支付成功，支付网关返回成功回调。
- 8) 系统更新订单状态为 `PAID`，实际扣减库存。
- 9) 跳转至“支付成功”页面，用户可以查看订单详情。

B. 异常场景 (Exception scenarios):

(1). 库存不足:

场景：用户提交订单时，商品实时库存小于购买数量。

处理：提交订单 API 校验失败，页面提示“库存不足，请调整数量”，阻止订单创建，不锁定库存。

(2). 支付失败:

场景：用户在支付页面点击支付，但支付网关返回失败（如余额不足、网络问题）。

处理：支付 API 返回失败信息。页面提示“支付失败：[具体原因]”，订单状态保持 `PENDING_PAYMENT`。用户可以返回该订单详情页尝试“重新支付”。

(3). 支付超时:

场景：用户创建订单后，超过 30 分钟（假设）未完成支付。

处理：一个定时任务扫描创建超过 30 分钟且状态为 `PENDING_PAYMENT` 的订单，将其状态更新为 `CANCELLED`，并释放锁定的库存。用户再次支付时会提示“订单已取消”。

(4). 商品已下架但仍存在于用户购物车中:

场景：运营人员将某商品下架后，该商品仍在用户的购物车列表中。

处理：

- a) 用户进入购物车页面时，后端返回商品信息并标记其状态。
- b) 前端将已下架的商品置灰，显示为“商品已下架”，勾选框禁用，无法选中进行结算。
- c) 用户点击“提交订单”时，后端再次校验订单中所有商品的 `status` 是否为 `ON_SALE`，如有下架商品，则拒绝创建订单并提示“部分商品已失效，请重新下单”。

C. 状态变更 (Status changes):

无状态 (购物车) → `PENDING_PAYMENT` (提交订单) → `PAID` (支付成功) / `CANCELLED` (支付超时/用户取消) / `PENDING_PAYMENT` (支付失败)

流程二：订单发货流程

A. 正常流程 (Normal Flow):

- (1). 运营人员在管理后台订单列表页，筛选状态为 `PAID` 的订单。
- (2). 运营人员点击订单后的“发货”按钮，进入发货页面。
- (3). 页面显示订单商品信息、收货地址。
- (4). 运营人员输入物流公司名称和物流单号。
- (5). 点击“确认发货”。
- (6). 后端更新订单状态为 `SHIPPED`，并保存物流信息。
- (7). 前端显示发货成功，订单列表状态更新。
- (8). 系统发送发货通知短信/推送给客户。

B. 异常场景 (Exception scenarios):

- (1). 物流单号格式错误：系统可对常见物流公司单号做正则校验，格式不匹配时提示“请输入有效的物流单号”，阻止发货。
- (2). 重复发货：订单状态已为 `SHIPPED` 或 `COMPLETED` 时，不应再显示“发货”按钮或后端应校验状态并拒绝。
- (3). 网络超时：运营点击“确认发货”后网络超时，应防止重复提交。前端按钮置灰，根据最终 API 返回结果提示“发货成功”或“发货失败，请重试”。

C. 状态变更 (Status changes):

`PAID` → `SHIPPED` (运营发货)

1.4 简化版 PRD

功能名称：管理端商品管理

1. 功能背景

当前系统没有商品录入功能，无法构建交易的基础。运营团队需要一套后台工具来创建、编辑和管理所有商品信息，包括不同规格 (`SKU`) 和库存，以保证前端商城有商品可卖，且信息准确。

2. 用户目标

作为运营人员，我希望能够：

- (1). 快速创建一个新商品，包括其基本信息、图片和多种规格。

- (2). 随时修改商品的价格、库存和上下架状态。
- (3). 方便地查看所有商品的列表。
- (4). 目标：确保商品信息准确无误，及时响应市场变化。

3. 页面字段

商品基本信息			
字段名	类型	规则/限制	是否必填
商品名称	文本输入框	最多 100 字	是
商品描述	富文本编辑器	支持图文，最多 5000 字	否
商品主图	图片上传	支持 jpg/png，单张 <5M，至少 1 张	是
商品详情图	多图上传	同上	否
商品类目	下拉选择	从预设类目树中选择	是

SKU (库存单位) 列表			
字段名	类型	规则/限制	是否必填
SKU 规格 (如：颜色/尺寸)	规格名+规格值	支持添加多组规格，笛卡尔积生成 SKU	是
商品编码 (SKU ID)	系统自动生成	不可编辑，用于唯一标识	自动
售价 (分)	数字输入框	整数，大于 0	是
市场价 (分)	数字输入框	整数，大于等于售价	否
库存数量	数字输入框	非负整数	是
SKU 图片	图片上传	针对不同规格可上传不同图片	否

状态控制			
字段名	类型	规则/限制	是否必填
上架/下架	开关按钮	控制商品是否在前端可见可售	是

4. 业务规则

- (1). 一个商品必须至少有一个 SKU。
- (2). 商品状态为“上架”时，前端才可见。当商品所有 SKU 库存均为 0 时，前端应显示“缺货”。
- (3). 商品下架后，不会从已存在的用户购物车中删除，而是置灰不可选。
- (4). 删除商品为逻辑删除（软删除），不在前端展示，但后台可查。

- (5). 编辑商品信息（如价格、库存）后，对用户端实时生效。已创建但未支付的订单不受价格变动影响，以创建订单时的快照为准。
- 5. 错误信息/异常处理
 - (1). 保存失败：网络问题或后端校验失败，提示“保存失败，请重试”。
 - (2). 必填项为空：提交时在对应字段下方标红提示“请填写商品名称”等。
 - (3). 图片上传失败：提示“图片上传失败，请检查格式/大小或重试”。
 - (4). 库存修改小于已锁定量：当运营尝试将库存修改为低于当前未支付订单锁定的库存总量时，系统应提示“操作失败，新库存数量不能低于已被预订的数量：XX”。
- 6. 验收标准
 - (1). 运营人员可以成功创建一个包含基本信息、主图和 1 个 SKU 的商品，并上架。在前端可以搜索并看到此商品。
 - (2). 运营人员可以成功创建一个包含“颜色”和“尺寸”两种规格的商品，系统自动生成 4 个 SKU，并能分别编辑价格和库存。
 - (3). 运营人员可以将一个已上架的商品进行“下架”操作，前端该商品详情页即显示“商品已下架”，无法购买。
 - (4). 运营人员修改某个 SKU 的价格后，前端该商品详情页的对应 SKU 价格能立即更新。
 - (5). 运营人员将某个 SKU 的库存修改为 0 后，前端该 SKU 变为“缺货”状态且无法选择或购买。

二、代码/技术理解

- 5. API 理解问答
 - (1). 可以展示信息：
 - 商品 ID、商品名称、商品状态（如“销售中”）、销售价（199.00 元）、原价（299.00 元）、库存数量（25 件）、销量（132 件）、商品图片列表。
 - (2). price: 19900 的展示：
 - 应在页面上展示为 199.00 元 或 ¥199.00。后端以“分”为单位返回是为了避免浮点数精度计算问题。
 - (3). status = OFF_SALE 的处理：
 - i 页面核心信息应正常展示。
 - ii “购买”或“加入购物车”按钮应置灰或不可点击。
 - iii 在旁边明显位置提示文案：“商品已下架”。
 - iv “立即购买”按钮不应跳转到确认订单页。
 - (4). 总库存为 0 的处理：
 - i 主“购买”或“立即购买”按钮应置灰。
 - ii 按钮文案可改为“缺货”或“暂时缺货”。
 - iii 详情页上可显示“库存不足，暂时无法购买”的提示。
 - (5). 选择了“黑色”SKU 的变化：
 - 前端应发起一个新的 API 请求，或者该 API 响应最初就包

含了所有 SKU 的数据。选择 SKU 时，页面上显示的单价、库存数量、以及可能不同的 SKU 图片都应切换为对应“黑色”SKU 的数据。

(6). 缺失字段：

- i skuList (SKU 列表): 商品详情页需要展示不同规格 (如颜色、尺寸) 及其对应的价格、库存、SKU 图片。缺少此字段，页面无法实现多规格选择。
- ii category (类目信息): 了解商品所在类目有助于前端展示“面包屑导航”、“相关推荐”等，对用户导航和 SEO 很重要。
- iii ratings 或 commentCount (评价信息): 评价是用户购买决策的关键因素。缺少此信息，页面无法展示商品口碑。
- iv shippingInfo (运费/物流信息): 用户在下单前需要知道是否需要付运费。缺少此信息，确认订单页逻辑不完整。
- v description (商品描述): 商品详情页下方的详细介绍区域，是转化用户的核心内容。API 中完全没有描述字段。

6. 简单逻辑阅读题

(1). 函数主要判断：

在创建订单前，判断用户购买的商品是否“有效”，即商品状态是否为销售中，以及所选 SKU 的库存是否足够。

(2). 无法提交订单的条件：

- i 商品状态不是 ON_SALE (例如已下架、已删除)。
- ii SKU 的当前库存少于用户想要购买的数量。

(3). 函数返回结果：

返回 "Insufficient stock" (库存不足)。

(4). 还应添加的校验：

- i 用户登录态校验: 判断用户是否已登录，未登录应先引导登录。
- ii 购买数量限制: 校验 buyCount 是否大于 0 且小于单次购买上限 (如最多 10 件)。
- iii 收货地址校验: 是否有默认收货地址，或是否填写了收货地址。
- iv 限购校验: 该商品是否限购 (如每人限购 2 件)，需要校验用户历史已购和当前购买总数。
- v 用户状态校验: 用户账号是否被禁用。

7. 订单状态理解

- (1). 初始状态: PENDING_PAYMENT (待支付)
- (2). 支付成功后: PAID (已支付)
- (3). 商家发货后: SHIPPED (已发货)
- (4). 支付时限内未支付: CANCELLED (已取消)
- (5). 可申请退款的状态: PENDING_PAYMENT (申请关单并退款), PAID (申请仅退款), SHIPPED (申请退货退款)。COMPLETED 也可申请售后，但通常有时间限制。

8. Bug/异常分析

可能原因:

(1). 数据不一致 (最常见):

商品详情页显示的库存数据来自缓存(如 Redis)或一个非实时的只读数据库从库(Read Replica)。而提交订单时, 后端直接校验的是主数据库(Master DB)中的真实库存。当缓存或从库还未同步主库的最新数据时, 就会造成差异。

(2). 并发购买:

用户 A 在商品详情页看到库存为 1, 在他点击提交订单的瞬间, 另一个用户 B 已经抢先完成了支付, 导致实际库存变为 0。当 A 的请求再到达后端时, 校验便会失败。

(3). 库存预扣未展示:

用户 A 浏览时库存为 1。之后用户 B 创建了订单, 系统锁定了这最后 1 件库存 (状态变为 PENDING_PAYMENT), 但前端详情页的库存查询逻辑因为某些原因没有减去这部分预扣库存, 导致显示仍为 1。

(4). SKU 混淆:

详情页展示的是整个商品的总库存 (例如所有颜色加起来共 1 件), 但用户选择了一个特定 SKU (如 “红色”), 提交订单时校验的是该 SKU 的库存 (可能为 0)。详情页应该展示所选 SKU 的库存。

调查步骤:

(1). 复现问题并获取关键信息:

请 QA 提供具体的用户 ID、商品 ID、SKU ID(如有)、出现问题的具体时间点。

(2). 日志分析:

查看提交订单时的后端 API 日志, 找到该用户对应请求的处理过程, 确认后端校验时的实时库存 (actual_stock) 到底是多少。

(3). 数据核对:

- i 查询主数据库 (Master DB), 确认当时该 SKU 的实际库存。
- ii 查询从数据库或缓存 (如 Redis), 确认当时返回给前端的库存值。
- iii 如果两者不一致: 根因定位为数据同步延迟或缓存更新策略问题。需要检查数据同步/缓存失效逻辑。
- iv 如果两者一致: 说明当时库存确实为 1, 问题不是数据不一致。

(4). 排查并发:

核对日志中是否有其他用户在同一秒或几毫秒内成功创建了同一 SKU 的订单。如果有，说明是并发问题。需要优化库存扣减逻辑（如使用乐观锁、悲观锁或 Redis 原子操作）。

(5). 排查库存预扣逻辑：

检查查询库存给前端的 SQL 或逻辑，看是否包含了状态为 PENDING_PAYMENT 订单的锁定量 (locked_stock)。正确的可售库存公式应为 total_stock - locked_stock。

(6). 结论与修复：

- i 若是缓存问题：修复缓存更新逻辑，确保库存变更后实时刷新缓存。
- ii 若是并发问题：在代码层面增加对同一商品/订单的分布式锁，确保串行处理。
- iii 若是预扣问题：统一所有地方的“可售库存”计算公式。

三、数据与指标

核心指标设计		
指标	含义	为何重要
日活跃用户 (DAU)	每日访问应用的独立用户数	衡量平台的整体用户规模和健康度。
商品详情页转化率	提交订单的用户数 / 商品详情页访客数 (或 UV)	衡量从“浏览”到“下单”整个商品及结账流程的吸引力与流畅度，是核心转化指标。
加购率	加入购物车的用户数 / 商品详情页访客数	衡量商品对用户的初步吸引力，比“下单”更上层的指标，能较早发现潜在问题。
支付成功率	支付成功的订单数 / 提交订单的订单数	直接衡量交易核心环节的成功与否。支付失败率高可能意味着支付网关集成有问题，或用户流程受阻。
库存周转天数	平均库存 / 日均销售商品数	衡量库存管理效率。周转天数过长说明库存积压，资金占用；过短则可能面临缺货风险。运营人员需重点关注。
平均订单价值 (AOV)	总 GMV / 总订单数	衡量用户的单次消费能力。可用于评估“满减”、“凑单”等促销策略的效果。
缺货率	发生缺货的 SKU 数 / 期间被请求的 SKU 数	衡量库存计划是否准确。缺货率高会导致用户体验下降和销售损失。
订单取消率	用户主动取消的订单数 / 总订单数	衡量用户购买意愿的坚定程度。取消率高可能源于冲动消费、支付流程障碍或商品信息与预期不符。

